

## Step 3 — Docker setup & validation

Every repository includes a `.helix/` folder in the repo root or in the [Docker Seeding] pull request with three files: `Dockerfile.helix`, `run-tests-eval.sh`, and `metadata.json`. Always use this — do **not** use external Docker images.

### Required system dependencies

The Dockerfile must install both `git` and `ca-certificates` as system dependencies. Without these, many build tools, package managers, and HTTPS operations will fail inside the container.

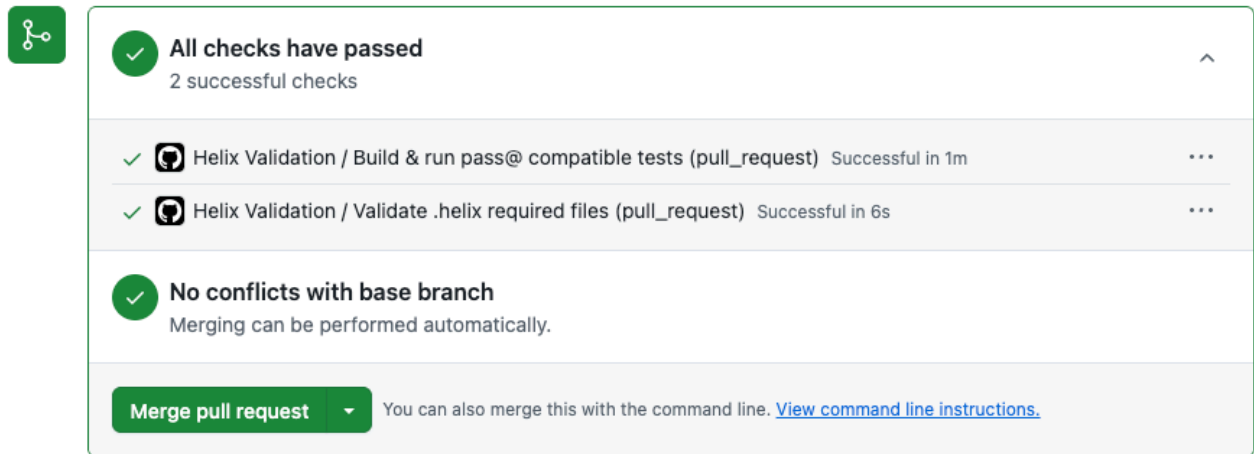
### Merge the Docker Seeding PR

If your repo has a [Docker Seeding] pull request, ensure all checks have passed and **merge it before proceeding**. The `.helix/` folder must be on `main` before you start building your solution.

Before merging, leave a review on the PR. This can be something simple — for example, "All validation checks passed." Then approve and merge.

### Don't skip this

If the Docker Seeding PR is still open, merge it now. Later steps assume that a `Dockerfile.helix` exists in the `.helix/` folder on the main branch.



The screenshot shows a GitHub pull request interface. On the left is a green icon with a white branching diagram. The main area is a light gray box with a green border. At the top, a green circle with a white checkmark is followed by the text "All checks have passed" and "2 successful checks". Below this are two check items, each with a green checkmark, a GitHub Actions icon, and text: "Helix Validation / Build & run pass@ compatible tests (pull\_request) Successful in 1m" and "Helix Validation / Validate .helix required files (pull\_request) Successful in 6s". At the bottom, a green circle with a white checkmark is followed by "No conflicts with base branch" and "Merging can be performed automatically." Below this is a green button with a white arrow pointing down and the text "Merge pull request". To the right of the button is the text "You can also merge this with the command line. [View command line instructions.](#)". At the bottom right of the box is the text "Still in progress? [Convert to draft](#)".

All checks passed — merge your Docker Seeding PR before continuing.

### Checkout main

Before building, make sure you're on `main`:

```
git checkout main
```

 Copy

## Build and validate

1

### Build the Docker image

From the repo root, build the test image:

```
docker build -f .helix/Dockerfile.helix -t helix-tests .
```

2

### Run tests to verify the environment

Run the RunScript to confirm the environment works:

```
docker run --entrypoint './.helix/run-tests-eval.sh' -t  
helix-tests
```

You don't need every test to pass — you just need to confirm that tests **can run**. If the environment builds and executes tests, you're in good shape.

3

### Try running specific tests

You can run individual tests to further verify the environment:

```
docker run --entrypoint './.helix/run-tests-eval.sh' -t  
helix-tests tests/unit-test1,tests/unit-test2
```

4

### Move on when confident

Once you've confirmed that the Docker environment can build and run tests successfully, proceed to Step 4.

## Validate pass-to-pass tests

Now investigate the **pass-to-pass tests** for this task.

### What are pass-to-pass tests?

Pass-to-pass tests are existing tests from the repository's test suite — they are **not created or modified** by the existing-solution branch. They must pass both before and after the feature code is applied, serving as regression checks to ensure new code doesn't break unrelated functionality.

**How to find them:** Compare the existing-solution branch to main. Any test files that were **not touched** by that branch but exercise related areas of the codebase are your pass-to-pass candidates.

Run them using the Docker image and run script to confirm they pass reliably:

```
docker run --entrypoint './.helix/run-tests-eval.sh' -t helix-  
tests tests/passing-test1,tests/passing-test2
```

 Copy

### Flaky tests mean a bad task

If any pass-to-pass tests fail intermittently or inconsistently, this task has flaky tests and is **not ideal**. Drop it and select a new task.

### Updating Docker during your golden solution

If you discover Docker issues (missing dependencies, broken test commands, etc.) while working on your `golden-solution` branch, do **not** fix them in your `[sol]` commit. Docker fixes must be merged into the default branch first so the validation system can use them.

1

#### Switch to the default branch

Leave your golden-solution branch and check out the default branch:  
`git checkout main && git pull`

2

#### Create a docker-golden-solution branch

Create a dedicated branch for the Docker fix:  
`git checkout -b docker-golden-solution`

3

#### Fix the Docker issue

Make the necessary changes to `.helix/Dockerfile.helix`, `.helix/run-tests-eval.sh`, or other Docker-related files.

4

#### Open a PR and merge

Push and open a PR into main. Wait for all **Helix Validation** checks to pass, then merge.

5

**Rebase your golden-solution branch**

Switch back to your golden-solution branch and rebase onto the updated default branch:

```
git checkout golden-solution && git rebase main
```

**Never modify Docker files in golden-solution commits**

Never modify `.helix/Dockerfile.helix` or `.helix/run-tests-eval.sh` in your golden-solution commits. The validation system uses the base commit's Docker setup, so Docker fixes must be merged into main first.

[Previous](#)[Next](#)